Text Mining: Alice's Adventures in Wonderland

Homework 3, Practical Part - A Needle in a Data Haystack (67978)

Mohammad Awawda

(a) The book we chose

We chose Alice's Adventures in Wonderland by Lewis Carroll, from Project Gutenberg (the text version). We picked it because we all know the story, so it is easy to check whether the results make sense. The source is https://www.gutenberg.org/cache/epub/11/pg11.txt. All the code sits in the code folder, and running text_mining_alice.py reproduces every table and figure below.

(b) Tokenizing and counting

First we cleaned the text. We then dropped the Project Gutenberg header and footer and the table of contents, and kept the story from the first chapter till the end. Then we tokenized it with nltk.word_tokenize on the lowercased text, and kept only the word tokens (letters), which drops the punctuation. word_tokenize also splits a possessive like alice's into alice and 's, so the name gets counted on its own without the additions. After that we counted the tokens with nltk.FreqDist and plotted the counts on a log-log scale, log frequency against log rank.

The plot shows the pattern we expected. A few words are very common and sit at the top ranks, while most words are rare and appear only once or twice, so they form a long tail. On the log-log scale the ponts drop in an almost straight line as the rank goes up.



Figure 1. Original token frequencies (log rank vs log frequency).

Item	Value
Input text	cleaned story text (header, footer and contents removed)
Tokenization	nltk.word_tokenize on lowercased text, keeping word tokens (letters only)
Counting	nltk.FreqDist
Total tokens	26,138
Unique tokens	2,480

(c) Top 20 tokens

These are the 20 most common tokens before we removed anything. Almost all of them are function words (the, and, to, a). The only content that shows up near the top is said and the name alice, which already tells us who the book is about.

Rank	Token	Count	Rank	Token	Count
1	the	1637	11	you	396
2	and	865	12	in	365
3	to	725	13	was	363
4	a	631	14	that	300
5	it	590	15	as	263
6	she	549	16	her	247
7	i	522	17	at	211
8	of	511	18	on	193
9	said	460	19	had	186
10	alice	396	20	all	179

(d) After removing stopwords

Next we removed English stopwords (the NLTK list) and repeated the same count and plot. Now the function words are gone and the top of the list is real content: said, alice, little, one, queen, king, turtle,

hatter, and so on. The log-log plot keeps the same shape, it just sits lower because we dropped a lot of very frequent words.

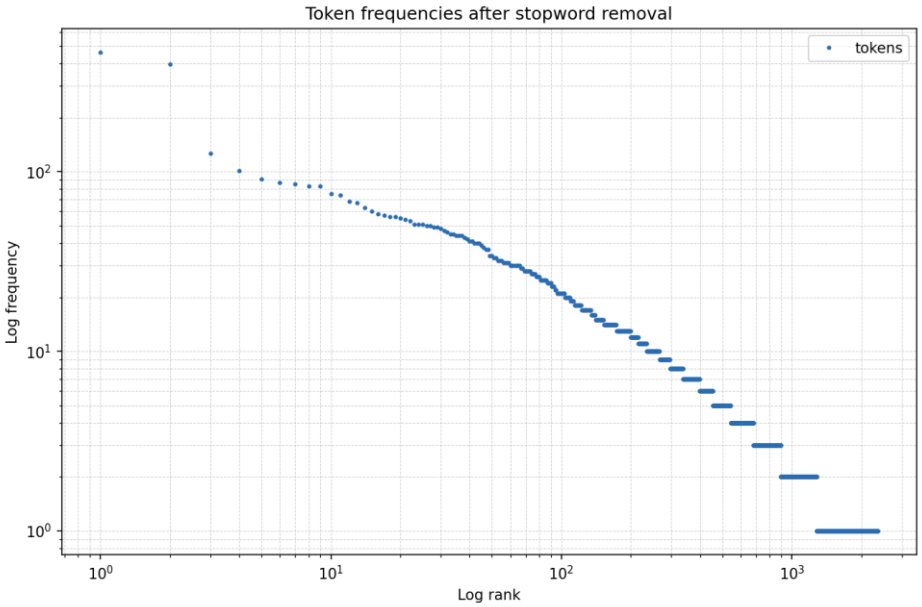


Figure 2. Token frequencies after stopword removal.

Rank	Token	Count	Rank	Token	Count
1	said	460	11	thought	74
2	alice	396	12	time	68
3	little	126	13	see	67
4	one	101	14	king	63
5	would	91	15	well	60
6	know	87	16	turtle	58
7	like	85	17	began	57
8	could	83	18	hatter	56
9	went	83	19	mock	56
10	queen	75	20	gryphon	55

(e) After stemming

Then we repeated (b) again, this time on the stemmed text. One important point: we stemmed the ORIGINAL tokens, not the stopword-removed ones, because the task says to repeat (b), and (b) runs on the full text. We used the NLTK Porter stemmer. Stemming cuts words to a root form, so forms like go, goes and going get counted together. The stems are not always real English words (for example was becomes wa, and alice becomes alic), which is normal for Porter.

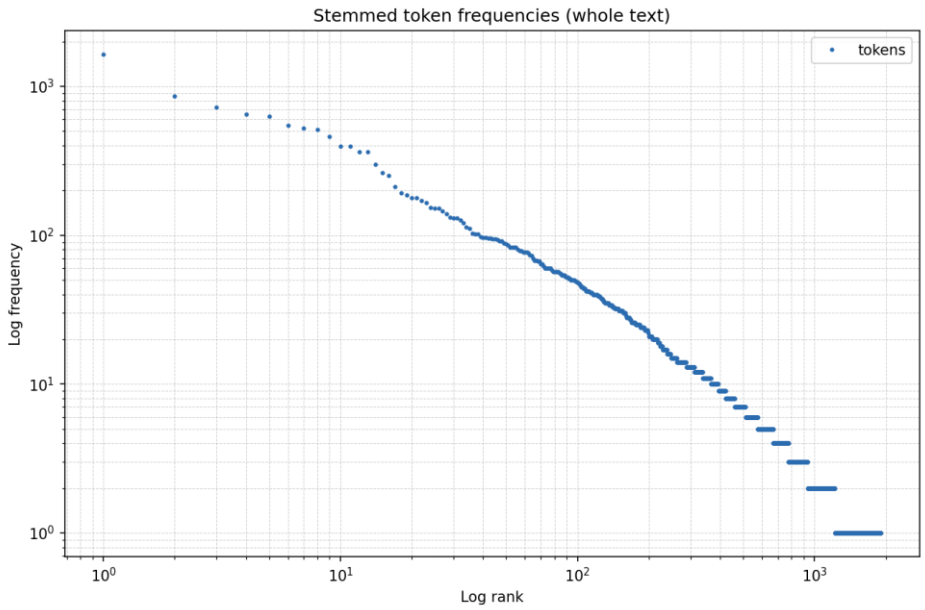


Figure 3. Stemmed token frequencies over the whole text.

Rank	Stem	Count	Rank	Stem	Count
1	the	1637	11	you	396
2	and	865	12	in	365
3	to	725	13	wa	363
4	it	647	14	that	300
5	a	631	15	as	263
6	she	549	16	her	251
7	i	522	17	at	211
8	of	511	18	on	193
9	said	460	19	had	186
10	alic	396	20	all	179

Comparing the three counts

The original text has 26,138 tokens and 2,480 unique tokens, and the top is all function words like the and and. After we remove stopwords we are left with 11,970 tokens and 2,353 unique tokens, and now the top words are content words like said and alice, so removing stopwords changes which words end up on top. Stemming keeps all 26,138 tokens (it changes word forms, it does not delete words), but it cuts the vocabulary from 2,480 down to 1,898 unique stems, because related forms collapse into one. So the two steps do different jobs: stopword removal changes which words rank highest, and stemming changes how many different words there are.

(f) A POS-tagging mistake

We ran POS tagging with NLTK on the cleaned text. We did not pick the example sentence by hand. Instead the code searches for a contradiction inside the tagging itself: a capitalized word that the tagger calls a common noun in one place, but calls a proper noun most of the time in the rest of the book. That word is almost surely a name, so the common noun tag is the wrong one. Here is the example it found automatically.

Item	Value
Selected sentence	I must have been changed for Mabel!

Item	Value
POS-tagged version	I/PRP must/MD have/VB been/VBN changed/VBN for/IN Mabel/NN !/.
Questionable token	Mabel
Tag NLTK gave it	NN (common noun)
Better tag	NNP (proper noun)

Mabel is a name here (Alice is scared she has been changed into another girl called Mabel), so it should be tagged NNP. The tagger gave it NN instead. We know it is a real mistake, not just our opinion, because the same tagger labels Mabel as a proper noun (NNP) 3 times in other sentences. So it is simply inconsistent in this one spot. This kind of error usually happens with names that the tagger has not seen often.

(g) Proper noun word cloud

We used the NNP and NNPS tags to pull out the proper nouns, and built a word cloud from their counts. We only did neutral cleaning (letters only, and drop very short leftovers). We did not remove any words by hand to make the cloud look nicer, so what you see is really what the tagger produced. We got 1,697 proper-noun tokens, 302 of them unique.



Figure 4. Word cloud of proper nouns (NNP and NNPS tags).

Rank	Proper noun	Count	Rank	Proper noun	Count
1	Alice	393	11	March	34
2	Queen	73	12	Hare	31
3	King	62	13	Mouse	30
4	Turtle	58	14	Caterpillar	27
5	Mock	56	15	Cat	26
6	Gryphon	55	16	White	22
7	Hatter	53	17	Well	20
8	Rabbit	42	18	Come	19
9	Dormouse	39	19	Oh	19
10	Duchess	35	20	Bill	16

It matches the book well. Alice is by far the biggest, which is right because the whole story follows her. After her come the main characters she meets: Queen, King, the Mock Turtle (which the tagger splits into Mock and Turtle), Gryphon, Hatter, Rabbit, Dormouse, Duchess, the March Hare (split into March and Hare), Mouse, Caterpillar and the Cat. So the cloud lines up with what we know about the story.

It is not perfect though, and we left the noise in on purpose. A few non-character words also show up, like Well, Come, Oh, How and No. These are usually the first word of a sentence, or a dialogue word, and the tagger marked them as proper nouns only because they are capitalized. This is the same tagging weakness we showed in part (f), so it is a nice honest example of why POS tags are useful but not fully reliable.

(h) Repeated-word regular expression

We wrote this regular expression to find two of the same word in a row, with optional punctuation between them:

```
\b([A-Za-z]{2,}|[AaIiOo])\b(?:[\s,;:!?.\-]|\u2014|\u2013)+\b\1\b
```

How it works: ([A-Za-z]{2,}|[AaIiOo]) captures the first word. The middle part allows whitespace and punctuation between the two copies, which covers cases like well well, so so and no, no. The backreference \1 forces the second word to be the same word, and we run it case-insensitive so No, no matches too.

Two things we were careful about. First, the word must be at least 2 letters, or one of the real one-letter words a, i, o. This stops single-letter junk: the stylised spelling e-e-evening in the Soup song would otherwise match as e e, which is not a real repeated word, so we skip it. Second, case-insensitive matching lets No, no match even though the two copies differ in case.

We found 34 matches, and they are not all the same thing, so we labeled each one. 30 of them are real repetition: emphasis (Down, down), stuttering (I I, of of, to to) and song lines (Twinkle, twinkle and Wow! wow). The other 4 are not real repetition. They are the same word that just happens to sit next to itself across a sentence or clause break (you: you, there. There, it: it, Two. Two). The task does allow punctuation between the words, so these technically match, but we mark them as sentence boundary so it is clear they are a different case and not true doubling.

All 34 matches

#	Match	Type	#	Match	Type
1	Down, down	repetition	18	it: it	sentence boundary
2	Down, down	repetition	19	Twinkle, twinkle	repetition
3	thump! thump	repetition	20	Twinkle, twinkle	repetition
4	you: you	sentence boundary	21	twinkle, twinkle	repetition
5	Dear, dear	repetition	22	Sh! sh	repetition
6	of—of	repetition	23	Two. Two	sentence boundary
7	there. There	sentence boundary	24	Hush! Hush	repetition
8	Prizes! Prizes	repetition	25	you! You	repetition
9	Pat! Pat	repetition	26	and—and	repetition
10	you, you	repetition	27	Tut, tut	repetition
11	if—if	repetition	28	No, no	repetition
12	it, it	repetition	29	Soup! Soup	repetition
13	I—I	repetition	30	Beautiful, beautiful	repetition
14	I—I	repetition	31	Beautiful, beautiful	repetition

#	Match	Type	#	Match	Type
15	No, no	repetition	32	unimportant—unimportant	repetition
16	Wow! wow	repetition	33	to—to	repetition
17	Wow! wow	repetition	34	No, no	repetition

A few matches with context

Match	Context from the book
Down, down	(Which was very likely true.) Down, down, down. Would the fall never come to an end?
thump! thump	did you ever eat a bat? when suddenly, thump! thump! down she came upon a heap of sticks
of—of	Are you-are you fond-of-of dogs? The Mouse did not answer
No, no	never one with such a neck as that! No, no! You're a serpent
there. There	placed along the course, here and there. There was no One, two, three (sentence boundary)